In [ ]:

```python
# read in data from excel file to dataframe
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
from scipy.stats import norm
from scipy.stats import t

#==================== Independent Constants ====================
class EnvConstants:
    """Environmental constants for atmospheric and crop conditions with units and descriptions.

    Attributes:
        P (float): Atmospheric pressure (kPa).
        k (float): von Karman's constant (none).
        esurface (float): Emissivity of crop (none).
        sigma (float): Stefan-Boltzmann constant (W m-2 K-4).
        KR (int): Parameter in Equ (24.2) (W m-2).
        KD1 (float): Parameter in Equ (24.6) (kPa-1).
        KD2 (float): Parameter in Equ (24.6) (kPa-2).
        TL (float): Parameter in Equ (24.4) and Equ (24.5) (K).
        T0 (float): Parameter in Equ (24.4) and Equ (24.5) (K).
        TH (float): Parameter in Equ (24.4) and Equ (24.5) (K).
        KM2 (float): Parameter in Equ (24.6) (mm-1).
        rho_a (float): Moist Air density (kg m-3).
        cp (float): Specific heat capacity of air at constant pressure (J kg-1 K-1).
        gc (float): Canopy cover factor (none).
        aT (float): Parameter in temperature stress factor (none).
    """
    P: float = 101.20  # Atmospheric pressure (kPa)
    k: float = 0.4  # von Karman's constant
    esurface: float = 0.95  # Emissivity of crop
    sigma: float = 5.67E-08  # Stefan-Boltzmann constant (W m-2 K-4)
    KR: int = 200  # Parameter in Equ (24.2)
    KD1: float = -0.307  # Parameter in Equ (24.6)
    KD2: float = 0.019  # Parameter in Equ (24.6)
    TL: float = 273.00  # Parameter in Equ (24.4) and Equ (24.5)
    T0: float = 293.00  # Parameter in Equ (24.4) and Equ (24.5)
    TH: float = 313.00  # Parameter in Equ (24.4) and Equ (24.5)
    KM2: float = -0.10  # Parameter in Equ (24.6)
    rho_a: float = 1.23  # Moist Air density
    cp: float = 1013.00  # Specific heat capacity of air
    gc: float = 1.00  # Canopy cover factor
    aT: float = 1.00  # Parameter in temperature stress factor

#==================== Crop Dependent Constants ====================

class ForestConstants:
    """Forest environmental constants with units and descriptions.

    Attributes:
        LAI (float): Leaf Area Index (none).
```

Partner: Fahim

Page 14: Forest Plots
Page 17: Grass Plots
Page 20 & 21: Tables
Page 22: ANSWERS TO QUESTIONS

```
        h (float): Canopy Height (m).
        a (float): Albedo (none).
        g0 (float): Canopy Specific Constant (mm s-1).
        SMo (float): Maximum soil moisture accessible to roots. Parameter in Equ (24.6) (mm).
        SMinit (float): Initial Root-accessible soil moisture (mm).
        S (float): Maximum Canopy Water Storage (mm).
        d (float): Zero plane displacement height (m).
        zm (float): Aerodynamic roughness of crop (m).
        KM1 (float): Parameter in Equ (24.6) (none).
    """
    LAI: float = 4.00  # Leaf Area Index
    h: float = 20.00  # Canopy Height
    a: float = 0.12  # Albedo
    g0: float = 15.00  # Canopy Specific Constant
    SMo: float = 80.00  # Maximum soil moisture accessible to roots
    SMinit: float = 40.00  # Initial Root-accessible soil moisture
    S: float = 4.00  # Maximum Canopy Water Storage
    zm: float = 22  # Aerodynamic roughness of crop
    LW_up_init: float = -350.00  # Initial upward longwave radiation (W m-2)
    KM1: float = 3.36E-04  # Parameter in Equ (24.6)
    d: float = np.NaN  # Zero plane displacement height
    z0: float = np.NaN  # Aerodynamic roughness of crop

class GrassConstants:
    """Grass environmental constants with units and descriptions.

    Attributes:
        LAI (float): Leaf Area Index (none).
        h (float): Canopy Height (m).
        a (float): Albedo (none).
        g0 (float): Canopy Specific Constant (mm s-1).
        SMo (float): Maximum soil moisture accessible to roots. Parameter in Equ (24.6) (mm).
        SMinit (float): Initial Root-accessible soil moisture (mm).
        S (float): Maximum Canopy Water Storage (mm).
        d (float): Zero plane displacement height (m).
        zm (float): Aerodynamic roughness of crop (m).
    """
    LAI: float = 2.00  # Leaf Area Index
    h: float = 0.12  # Canopy Height
    a: float = 0.23  # Albedo
    g0: float = 30.00  # Canopy Specific Constant
    SMo: float = 40.00  # Maximum soil moisture accessible to roots
    SMinit: float = 10.00  # Initial Root-accessible soil moisture
    S: float = 2.00  # Maximum Canopy Water Storage
    zm: float = 2  # Aerodynamic roughness of crop
    LW_up_init: float = -319.00  # Initial upward longwave radiation (W m-2)
    KM1: float = 1.87E-02  # Parameter in Equ (24.6)
    d: float = np.NaN  # Zero plane displacement height
    z0: float = np.NaN  # Aerodynamic roughness of crop

#==================== Model forcing data ==========================
excel_file = "Lab1.xlsx"
# Read the data from the Excel file, skipping rows so that headers start from row 11 (indexing starts from 0)
forest_data = pd.read_excel(excel_file, header=0, sheet_name = "Forest", skiprows=10)
grass_data = pd.read_excel(excel_file, sheet_name="Grass", header=0, skiprows=10)
```

```
c:\Users\adunw\miniconda3\envs\EcoHydro\Lib\site-packages\openpyxl\worksheet\_read_only.py:81: UserWarning: Unknown extension is not supported and will be removed
  for idx, row in parser.parse():
c:\Users\adunw\miniconda3\envs\EcoHydro\Lib\site-packages\openpyxl\worksheet\_read_only.py:81: UserWarning: Unknown extension is not supported and will be removed
  for idx, row in parser.parse():
```

In [ ]:
```python
# Part A: Aerodynamic Parameterization.

def calc_d(h, LAI):
    """
    Calculate zero plane displacement height using constants from either ForestConstants or Grasscrop_constants.
    (TH Eq 22.2)
    :param h: Canopy height (m). Crop constant.
    :param LAI: Leaf Area Index (none). Crop constant.
    :return: Zero plane displacement height (m).
    """
    return 1.1 * h * np.log(1 + (LAI / 5)**0.25)  # TH Eq 22.2

def calc_zo(h, d):
    """
    Calculate aerodynamic roughness of crop. (TH Eq 22.3, 22.4)
    :param h: Canopy height (m). Crop constant.
    :param d: Zero plane displacement height (m).
    :param LAI: Leaf Area Index (none). Crop constant.
    :return: Aerodynamic roughness of crop (m).
    """

    return 0.3 * h * (1 - d / h)  # TH Eq 22.4


def calc_ra(d, zm, um, z0):
    """
    Calculate aerodynamic resistance using constants from Environmentalcrop_constants.
    (TH Eq 22.9)
    :param zm: Measurement height for wind speed (m). Environmental constant.
    :param d: Zero plane displacement height (m).
    :param zo: Aerodynamic roughness of crop (m).
    :param um: Wind speed at measurement height (m s-1).
    :param k: von Karman's constant (none). Environmental constant.
    :return: Aerodynamic resistance (s m-1).
    """
    k = EnvConstants.k
    return 1 / (k**2 * um) * np.log((zm - d) / z0) * np.log((zm - d) /(z0/10))  # TH Eq 22.9
```

In [ ]:
```python
# Part B: Surface Resistance Parameterization

def calc_gR(SW_in):
    """
    Calculate resistance to soil heat flux (Eq. 24.2 in TH)
    :param SW_in: Incoming shortwave radiation (W m-2).
    :param KR: Parameter in Equ (24.2) (W m-2). Environemntal constant.
    :return: Resistance to soil heat flux (none).
    """
    KR = EnvConstants.KR
    gR =  SW_in * (1000 + KR) / (1000 * (SW_in + KR)) # Eq. 24.2 TH
    return max(gR, 0.)
```

```python
def calc_gD(D):
    """
    Calculate resistance to vapor pressure deficit (Eq. 24.3 in TH)
    :param D: Vapor pressure deficit (kPa).
    :param KD1: Parameter in Equ (24.6) (kPa-1). Environemntal constant.
    :param KD2: Parameter in Equ (24.6) (kPa-2). Environemntal constant.
    :return: Resistance to vapor pressure deficit (none).
    """
    KD1 = EnvConstants.KD1
    KD2 = EnvConstants.KD2
    gD = 1 + KD1 * D + KD2 * D**2 # Eq. 24.3 TH
    return max(gD, 0.)

def calc_gT(T):
    """
    Calculate resistance to temperature (Eq. 24.4 in TH)
    :param T: Temperature (K).
    :param TL: Parameter in Equ (24.4) and Equ (24.5) (K). Environemntal constant.
    :param T0: Parameter in Equ (24.4) and Equ (24.5) (K). Environemntal constant.
    :param TH: Parameter in Equ (24.4) and Equ (24.5) (K). Environemntal constant.
    :param aT: Parameter in temperature stress factor (none). Environemntal constant.
    :return: Resistance to temperature (none).
    """
    TL = EnvConstants.TL
    T0 = EnvConstants.T0
    TH = EnvConstants.TH
    aT = EnvConstants.aT
    gT = ((T - TL) * (TH - T)**aT) / ((T0 - TL) * (TH - T0)**aT) # Eq. 24.4 TH, problem statemnt B.1
    return max(gT, 0.)

def calc_gSM(SM, SMo, KM1):
    """
    Calculate resistance to soil moisture flux (Eq. 24.6 in TH)
    :param SM: Soil moisture (mm).
    :param SMo: Maximum soil moisture accessible to roots. Parameter in Equ (24.6) (mm).
    :param KM1: Parameter in Equ (24.6) (none). Crop constant.
    :param KM2: Parameter in Equ (24.6) (mm-1). Environemntal constant.
    :return: Resistance to soil moisture flux (none).
    """
    KM2 = EnvConstants.KM2

    gSM = 1 - KM1 * np.exp(KM2 * (SM - SMo))  # Eq. 24.6 TH
    return max(gSM, 0.)

def calc_gs(gR, gD, gT, gSM, g0):
    """
    Calculate total surface conductance (Eq. 24.1 in TH)
    :param gR: Resistance to soil heat flux (none).
    :param gD: Resistance to vapor pressure deficit (none).
    :param gT: Resistance to temperature (none).
    :param gSM: Resistance to soil moisture flux (none).
    :param g0: Canopy Specific Constant (mm s-1). Crop constant.
    :return: Total surface conductance (mm s-1).
    """
    gs = (gR * gD * gT * gSM * g0) # Eq. 24.1 TH
```

```
    return max(gs, 10**-4)

def calc_rs(gs):
    """
    Calculate total surface resistance (Eq. 24.1 in TH)
    :param gs: Total surface conductance (mm s-1).
    :return: Total surface resistance (s mm-1).
    """
    rs = 1 / gs # Eq. 24.1 TH
    rs *= 10**3  # convert to s m-1
    return min(rs, 10**6)

def calc_esat(T):
    """
    Calculate saturation vapor pressure from temperature (Eq. 2.17 in TH)
    :param T: Temperature (C).
    :return: Saturation vapor pressure (kPa).
    """
    esat = 0.6108 * np.exp(17.27 * T / (T + 237.3)) # Eq. 2.17 TH
    return esat

def calc_e(q):
    """
    Calculate vapor pressure from specific humidity and pressure (Eq. 2.9 in TH)
    :param q: Specific humidity (kg kg-1).
    :param P: Atmospheric pressure (kPa). Environemntal constant.
    :return: Vapor pressure (kPa).
    """
    P = EnvConstants.P
    e = P * q / 0.622 # Eq 2.9 TH
    return e

def calc_D(esat, e):
    """
    Calculate vapor pressure deficit (Eq. 2.10 in TH)
    :param esat: Saturation vapor pressure (kPa).
    :param e: Vapor pressure (kPa).
    :return: Vapor pressure deficit (kPa).
    """
    D = esat - e # Eq 2.10 TH
    return D
```

In [ ]:

```
#Part C: Radiation Parameterization

def calc_Ts(Ta, H, ra):
    """
    Calculate surface temperature (Eq. 21.30 in TH)
    :param Ta: Air temperature (K).
    :param H: Sensible heat flux (W m-2).
    :param ra: Aerodynamic resistance (s m-1).
    :param rho_a: Moist Air density (kg m-3). Environmental Constant.
    :param cp: Specific heat capacity of air (J kg-1 K-1). Environmental Constant.
    :return: Surface temperature (K).
    """
    cp = EnvConstants.cp
    rho_a = EnvConstants.rho_a
```

```
    Ts = Ta + (H *ra / rho_a / cp) # Eq 21.30 TH
    return Ts

def calc_LW_up(Ts_1, Ts_2):
    """
    Calculate long wave radiation from surface temperature and emissivity. (Eq 5.19 in TH)
    :param T_s: Surface temperature (K).
    :param emissivity: Emissivity of crop (none). Environmental constant.
    :param sigma: Stefan-Boltzmann constant (W m-2 K-4). Environmental constant.
    :return: Long wave radiation from surface (W m-2).
    """
    sigma = EnvConstants.sigma
    emissivity = EnvConstants.esurface
    LW_up = - emissivity * sigma * (Ts_1**4 + Ts_2**4)/2 # Modified Eq 5.19 TH (Problem statement C.2)
    return LW_up

def calc_Rn(SW_in, a, LW_up, LW_down):
    """
    Calculate net radiation (Eq 5.28 in TH)
    :param SW_in: Incoming shortwave radiation (W m-2).
    :param a: Albedo (none). Crop Constant.
    :param LW_up: Long wave radiation from surface (W m-2).
    :param LW_down: Long wave radiation from atmosphere (W m-2).
    :return: Net radiation (W m-2).
    """
    LW_net = LW_down + LW_up
    Rn = (1 - a) * SW_in + LW_net # Eq 5.28 TH
    return Rn
```

In [ ]:

```
#Part D.1 Canopy Water Balance Parameterization (Includes Interception)

def calc_delta(T, e_sat):
    """
    Calculate gradient of the saturation vapor pressure curve from saturation vapor pressure and temperature
    (Eq 2.18 in TH)
    e_sat: saturation vapor pressure in kPa
    T: temperature in degrees Celsius
    return: slope of the saturation vapor pressure curve in kPa/°C
    """

    delta = 4098 * e_sat / (T + 237.3)**2 # Eq 2.18 TH, problem statement D.1.1
    return delta

def calc_LH(T):
    """
    Calculate latent heat of vaporization from temperature (Eq 2.1 in TH)
    T: temperature in degrees Celsius
    return: latent heat of vaporization (lambda) in J/kg
    """
    LH = (2.501 - 0.002361 * T )* 10**6 # Eq 2.1 TH, problem statement D.1.1
    return LH

def calc_psy_const(LH):
    """
    Calculate psychrometric constant (Eq 2.25 in TH)
    c_p: specific heat of dry air at constant pressure in J/kg/K
```

```
    P: pressure in kPa
    LH: latent heat of vaporization in J/kg
    return: psychrometric constant in kPa/°C
    """
    cp = EnvConstants.cp # Specific heat capacity of air at constant pressure in J/kg/K
    P = EnvConstants.P
    psychrometric_constant = cp * P / (0.622 * LH) # Eq. 2.25 TH, problem statement D.1.1
    return psychrometric_constant

def calc_Cinterim(p, Cfinal):
    """
    Calculate interim canopy drainage (D.1.6 in problem statement)
    :param p: precipitation in mm
    :param Cfinal: final canopy storage in mm
    return: interim canopy storage in mm
    """
    Cinterim = p + Cfinal # Given in problem statement D.1.6
    # Ensure postive value
    return max(Cinterim, 0)

def calc_Cactual(Cinterim, S):
    """
    Calculate actual canopy storage for each row.
    :param Cinterim: interim canopy storage in mm
    :param S: maximum canopy storage in mm
    :return: actual canopy storage in mm
    """
    return min(Cinterim, S)  # Ensure Cactual is not greater than S

def calc_Dcanopy(Cinterim, S):
    """
    Calculate canopy drainage for each row.
    :param Cinterim: interim canopy storage in mm
    :param S:  maximum canopy storage in mm
    :return: canopy drainage in mm
    """
    Dcanopy = max(Cinterim - S, 0)  # Ensure Dcanopy is not negative
    return Dcanopy


def calc_Cfinal(lambdaEi, Cactual, LH):
    """
    Calculate final canopy storage
    :param lambdaEi: evaporation rate of intercepted rainfall with free water canopy in W m-2
    :param Cactual: actual canopy storage in mm
    :param LH: latent heat of vaporization in J/kg
    return: final canopy storage in mm
    """
    m_to_mm = 1000
    rho_w = 1000 # Density of water in kg/m^3
    lambdaEi_mm = lambdaEi * 3600 / LH / rho_w * m_to_mm # Convert from W m-2 to mm
    Cfinal = max(Cactual - lambdaEi_mm,0) # Problem statement D.1.9
    return Cfinal

def calc_lambdaEi(delta, ra, Rn, psy_const, D, C, S):
```

```
    """
    Calculate evaporation rate of intercepted rainfall with free water canopy (Eq 22.14 in TH)
    :param delta: slope of the saturation vapor pressure curve in kPa/°C
    :param ra: aerodynamic resistance in s/m
    :param Rn: net radiation in W/m^2
    :param psy_const: psychrometric constant in kPa/°C
    :param rho_a: moist air density in kg/m^3. Environmental constant.
    :param cp: specific heat capacity of air at constant pressure in J/kg/K. Environmental constant.
    :param D: vapor pressure deficit in kPa
    return: evaporation rate of intercepted rainfall with free water canopy in W m-2
    """
    A = Rn # Available energy = net radiation problem statement D.1.8
    Dref = D # VPD at reference level above canopy in kPa
    rho_a = EnvConstants.rho_a # Moist Air density
    cp = EnvConstants.cp # Specific heat capacity of air at constant pressure in J kg-1 K-1
    lambdaEi = C / S * (delta * A + (rho_a * cp * Dref / ra) ) / (delta + psy_const) # Eq 22.14 TH, problem statement D.1.4
    return lambdaEi

def calc_lambdaET(delta, Rn, D, psy_const, rs, ra):
    """
    Calculate transpiration rate (Eq 22.18 in TH)
    :param delta: slope of the saturation vapor pressure curve in kPa/°C
    :param Rn: net radiation in W/m^2
    :param D: vapor pressure deficit in kPa
    :param psy_const: psychrometric constant in kPa/°C
    :param ra: aerodynamic resistance in s/m
    return: transpiration rate in W m-2
    """
    A = Rn # Available energy = net radiation problem statement D.1.8
    Dref = D # VPD at reference level above canopy in kPa
    rho_a = EnvConstants.rho_a
    cp = EnvConstants.cp
    lambdaET = (delta * A + (rho_a * cp * Dref / ra) ) / (delta + psy_const * (1 + rs / ra)) # Eq 22.18 TH, problem statement D.1.5
    return lambdaET

def calc_lambdaE(lambdaEi, C, S, lambdaET):
    """
    Calculate total evaporation rate (Eq 22.17 in TH)
    :param lambdaEi: evaporation rate of intercepted rainfall with free water canopy in W m-2
    :param C: canopy storage in mm
    :param S: maximum canopy storage in mm
    :param lambdaET: transpiration rate in W m-2
    return: total evaporation rate in W m-2
    """
    lambdaE = lambdaEi + (1 - C/S) * lambdaET # Eq 22.17 TH, problem statement D.1.2
    return lambdaE

def calc_H(R_n, lambdaE):
    """
    Calculate turbulent sensible heat flux from net radiation, evaporation rate, and heat flux into the ground.
    (D.1.1 & D.1.12 in problem statement)
    :param R_n: Net radiation (W m-2).
    :param lambdaE: Evaporation rate (W m-2).
    return: Sensible heat flux (W m-2).
    """
```

```
    H = R_n - lambdaE # Given in problem statement D.1.1 & D.1.12
    return H
```

In [ ]:
```
# Part E Soil Water Balance Parameterization

def calc_SMnew(Dcanopy, Cactual, S, lambdaET, LH, SMo, SMlast):
    """
    Calculate new soil moisture (E.1 & E.2 in problem statement)
    :param SMlast: Previous soil moisture (mm).
    :param Dcanopy: Canopy water storage change (mm).
    :param Cactual: Actual canopy water storage (mm).
    :param S: Maximum canopy water storage (mm). Crop constant.
    :param lambdaET: Evapotranspiration (W m-2).
    :param LH: Latent heat of vaporization (J/kg). Environmental constant.
    :param SMo: Maximum soil moisture accessible to roots (mm). Crop constant.
    :return: New soil moisture (mm).
    """
    m_to_mm = 1000
    rho_w = 1000 # Density of water in kg/m^3
    lambdaET_mm = lambdaET * 3600 / LH / rho_w * m_to_mm # Convert from W m-2 to mm
    SMnew = Dcanopy - (1 - Cactual / S) * lambdaET_mm # problem statement E.1
    # Ensure SMnew does not exceed soil moisture holding capacity
    SMnew = SMnew + SMlast

    SMnew = min(SMnew, SMo) #problem statement E.2, make sure its not greater than moisture holding capacity
    return max(SMnew,0) # Ensure SMnew is not negative
```

In [ ]:
```
def setup_df(data):
    data['ra'] = np.nan
    data['e'] = np.nan
    data['esat(T)'] = np.nan
    data['D'] = np.nan
    data['Tk'] = np.nan
    data['gR'] = np.nan
    data['gD'] = np.nan
    data['gT'] = np.nan
    data['gSM'] = np.nan
    data['SM'] = np.nan
    data['gs'] = np.nan
    data['rs'] = np.nan
    data['LW_up'] = np.nan
    data['Rn'] = np.nan
    data['LH'] = np.nan
    data['delta'] = np.nan
    data['psy_const'] = np.nan
    data['Cinterim'] = np.nan
    data['Cactual'] = np.nan
    data['Cfinal'] = np.nan
    data['Dcanopy'] = np.nan
    data['lambdaEi (C=Cactual)'] = np.nan
    data['lambdaET'] = np.nan
    data['lambdaE'] = np.nan
    data['H'] = np.nan
    data['Ts'] = np.nan
```

```
    return data
```

In [ ]:

```
def run_model(data, crop_constants):
    """
    Run the model on the given dataset using functions defined above.
    Write the results to a csv file.

    :param data: DataFrame containing either crop data.
    :param crop_constants: Dictionary of constants specific to the crop.
    """
    S = crop_constants.S # Maximum Canopy Water Storage (mm)
    h = crop_constants.h # Canopy Height (m)
    LAI = crop_constants.LAI # Leaf Area Index (none)
    a = crop_constants.a # Albedo (none)
    g0 = crop_constants.g0 # Canopy Specific Constant (mm s-1)
    SMo = crop_constants.SMo # Maximum soil moisture accessible to roots. Parameter in Equ (24.6) (mm)
    SMinit = crop_constants.SMinit # Initial Root-accessible soil moisture (mm)
    zm = crop_constants.zm
    KM1 = crop_constants.KM1
    # Setup dataframe in following roder: ra    e       esat(T) D       T       gR      gD      gT      gSM     SMlast  gs      rs      Lu      Rn      l       D       g       Cinterim        Cactu
    # in one line
    data = setup_df(data)
    data['q'] = data['q'] / 1000 # Convert specific humidity from g/kg to kg/kg
    data['Tk'] = data['Ta'] + 273.17 # Convert temperature from Celsius to Kelvin

    # Part A: Aerodynamic Parameterization
    # data['d'] = calc_d(h, LAI) # A.1
    # data['z0'] = calc_zo(h, data['d']) # A.1
    crop_constants.d = calc_d(h, LAI) # A.1
    crop_constants.z0 = calc_zo(h, crop_constants.d) # A.1
    data['ra'] = calc_ra(crop_constants.d, zm, data['um'], crop_constants.z0) # A.2

    # Part B: Surface Resistance Parameterization
    data['esat(T)'] = calc_esat(data['Ta'])
    data['e'] = calc_e(data['q'])
    data['D'] = calc_D(data['esat(T)'], data['e'])

    # D.1 Canopy Water Balance Parameterization
    data['delta'] = calc_delta(data['Ta'], data['esat(T)']) #D.1.1
    data['LH'] = calc_LH(data['Ta']) #D.1.1
    data['psy_const'] = calc_psy_const(data['LH']) #D.1.1

    #set first two values of LW_up to initial value
    data.at[0, 'LW_up'] = crop_constants.LW_up_init
    data.at[1, 'LW_up'] = crop_constants.LW_up_init
    data.at[0, 'SM'] = SMinit
    data.at[0, 'Cfinal'] = 0

    for index in range(len(data)):

        # Current values for the row
        LH = data.at[index, 'LH'] # Latent heat of vaporization
        delta = data.at[index, 'delta'] # Slope of the saturation vapor pressure curve
        ra = data.at[index, 'ra'] # Aerodynamic resistance
        psy_const = data.at[index, 'psy_const']  # Psychrometric constant
```

```
        D = data.at[index, 'D'] # Current vapor pressure deficit
        p = data.at[index, 'p']  # Current precipitation
        Tk = data.at[index, 'Tk']  # Current temperature
        SW_in = data.at[index, 'SW_in']  # Incoming shortwave radiation
        LW_down = data.at[index, 'LW_down']  # Long wave radiation from atmosphere

        if index == 0:
            Cfinal_last = 0
            SMlast = SMinit

        #Part C: Radiation Parameterization
        if index > 1:

            Ts_1 = data.at[index - 1, 'Ts']
            Ts_2 = data.at[index - 2, 'Ts']
            data.at[index, 'LW_up'] = calc_LW_up(Ts_1, Ts_2)

        LW_up = data.at[index, 'LW_up']  # Upward longwave radiation
        Rn = calc_Rn(SW_in, a, LW_up, LW_down)


        gR = calc_gR(SW_in)
        gD = calc_gD(D)
        gT = calc_gT(Tk)
        gSM = calc_gSM(SMlast, SMo, KM1)
        gs = calc_gs(gR, gD, gT, gSM, g0)
        rs = calc_rs(gs)

        # Part D.1 Canopy Water Balance Parameterization
        Cinterim = calc_Cinterim(p, Cfinal_last)
        Cactual = calc_Cactual(Cinterim, S)
        Dcanopy = calc_Dcanopy(Cinterim, S)
        lambdaEi_Cactual = calc_lambdaEi(delta, ra, Rn, psy_const, D, Cactual, S)
        lambdaET = calc_lambdaET(delta, Rn, D, psy_const, rs, ra)
        lambdaE = calc_lambdaE(lambdaEi_Cactual, Cactual, S, lambdaET)
        H = calc_H(Rn, lambdaE)
        SM = calc_SMnew(Dcanopy, Cactual, S, lambdaET, LH, SMo, SMlast)
        Cfinal = calc_Cfinal(lambdaEi_Cactual, Cactual, LH)
        Ts = calc_Ts(Tk, H, ra)
        # Update the DataFrame with the calculated values for the current row

        data.at[index, 'gR'] = gR
        data.at[index, 'gD'] = gD
        data.at[index, 'gT'] = gT
        data.at[index, 'gSM'] = gSM
        data.at[index, 'gs'] = gs
        data.at[index, 'rs'] = rs
        data.at[index, 'SM'] = SM
        data.at[index, 'LW_up'] = LW_up
        data.at[index, 'Rn'] = Rn  # Net radiation
        data.at[index, 'Cinterim'] = Cinterim # Problem statement D.1.6
        data.at[index, 'Cactual'] = Cactual  # Problem statement D.1.7
        data.at[index, 'Cfinal'] = Cfinal # Problem statement D.1.9
        data.at[index, 'Dcanopy'] = Dcanopy # Problem statement D.1.7
        data.at[index, 'lambdaEi (C=Cactual)'] = lambdaEi_Cactual # Problem statement D.1.8
        data.at[index, 'lambdaET'] = lambdaET # Problem statement D.1.10
```

```
        data.at[index, 'lambdaE'] = lambdaE # Problem statement D.1.11
        data.at[index, 'H'] = H # Problem statement D.1.12
        data.at[index, 'Ts'] = Ts

        SMlast = SM
        Cfinal_last = Cfinal

    data.to_csv('processed_forest_data.csv', index=False)


forest_constants = ForestConstants()
grass_constants = GrassConstants()
# Example usage with the provided datasets
run_model(forest_data, forest_constants)
run_model(grass_data, grass_constants)

# After processing, you can export the modified datasets to CSV files
forest_data.to_csv('processed_forest_data.csv', index=False)
grass_data.to_csv('processed_grass_data.csv', index=False)
```

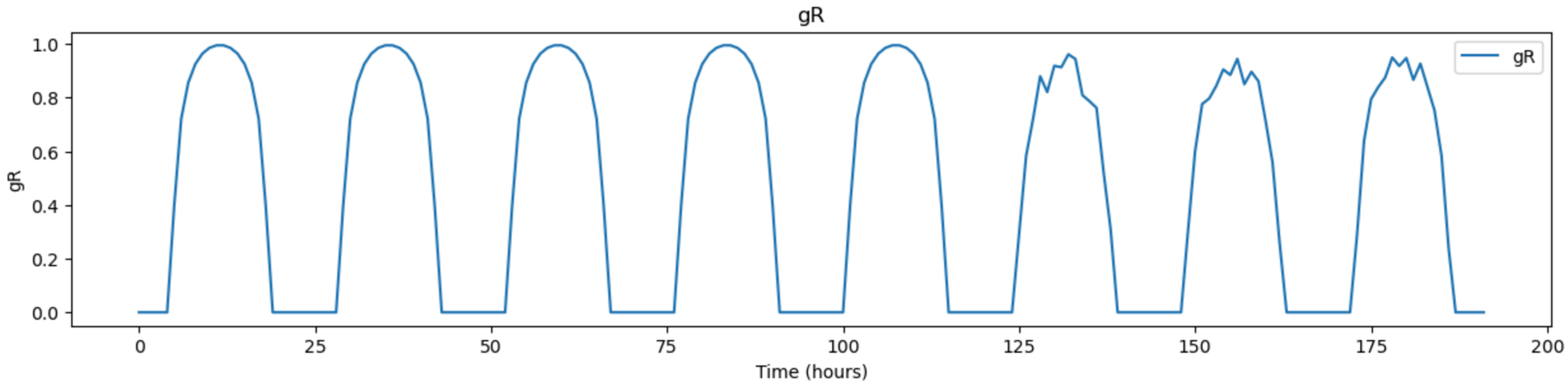
In [ ]:

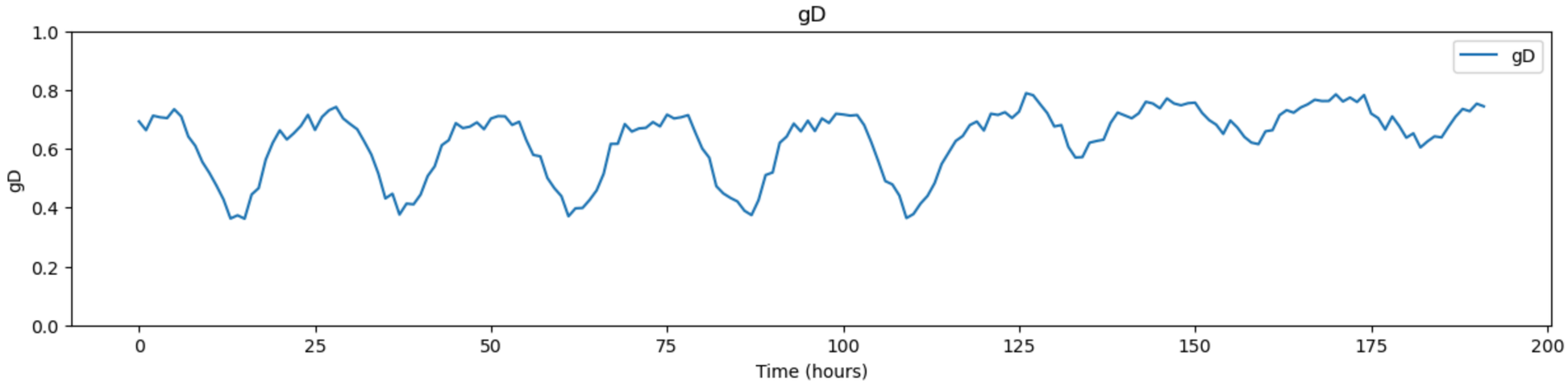
In [ ]:
```
#Plot the data
def plot_data(df):
    """
    Plot the processed data from the forest dataset.
    :param df: DataFrame containing the processed forest data.
    """
    # Plot gR wide layout with y min of 0
    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['gR'], label='gR')
    ax.set_title('gR')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('gR')
    ax.legend()
    plt.show()

    #Plot gD wide layout with y min of 0
    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['gD'], label='gD')
    ax.set_title('gD')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('gD')
    ax.set_ylim(0, 1)
    ax.legend()
    plt.show()

    #Plot gT wide layout
    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['gT'], label='gT')
    ax.set_title('gT')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('gT')
    ax.set_ylim(0, 1)
    ax.legend()
```

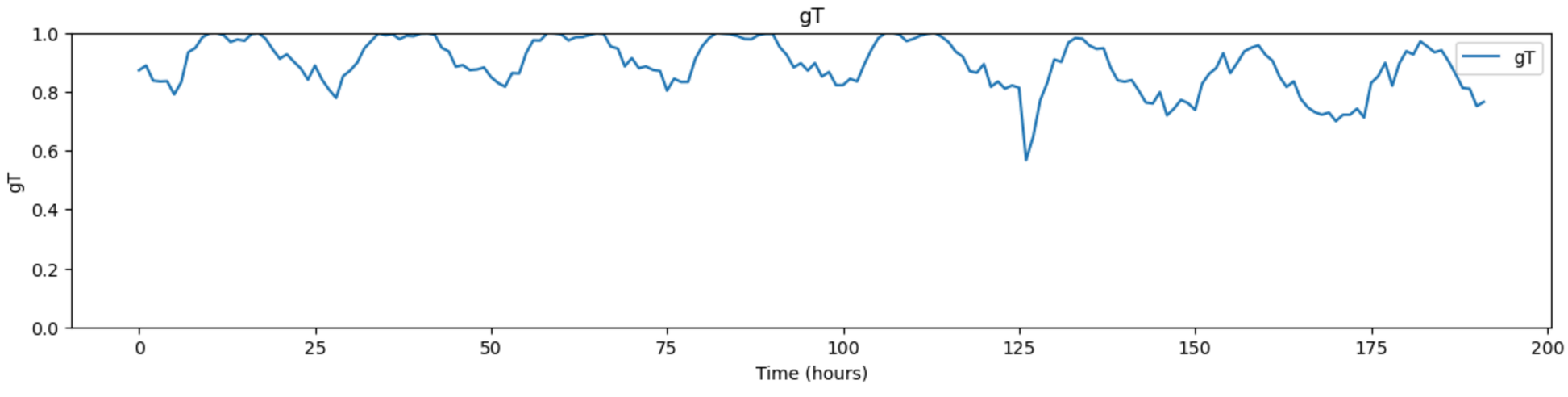
```
    plt.show()

    #Plot Hourly values of the calculated net radiation, total latent heat flux, and sensible  heat flux (all in W m-2) on the same graph
    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['Rn'], label='Net Radiation')
    ax.plot(df['lambdaE'], label='Total Latent Heat Flux')
    ax.plot(df['H'], label='Sensible Heat Flux')
    ax.set_title('Radiation Budget')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('W m-2')
    ax.legend()
    plt.show()

    # Plot Hourly values of the calculated total latent heat flux and the portion of the latent heat flux that originates from the evaporation of intercepted water (both in W m-2).

    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['lambdaE'], label='Total Latent Heat Flux')
    ax.plot(df['lambdaEi (C=Cactual)'], label='Intercepted Water Evaporation')
    ax.set_title('Latent Heat Flux')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('W m-2')
    ax.legend()
    plt.show()

    # Plot Hourly values of the precipitation and the canopy storage C (both in mm)
    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['p'], label='Precipitation')
    ax.plot(df['Cfinal'], label='Canopy Storage')
    ax.set_title('Precipitation and Canopy Storage')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('mm')
    ax.legend()
    plt.show()

    # Plot Hourly values of the canopy drainage and the value of SM (both in mm)
    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['Dcanopy'], label='Canopy Drainage')
    ax.plot(df['SM'], label='Soil Moisture')
    ax.set_title('Canopy Drainage and Soil Moisture')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('mm')
    ax.legend()
    plt.show()

    # Plot Hourly values of the soil moisture stress function gM (no units)
    fig, ax = plt.subplots(figsize=(15, 3))
    ax.plot(df['gSM'], label='Soil Moisture Stress')
    ax.set_title('Soil Moisture Stress')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('dimensionless')
    ax.legend()
    plt.show()
```

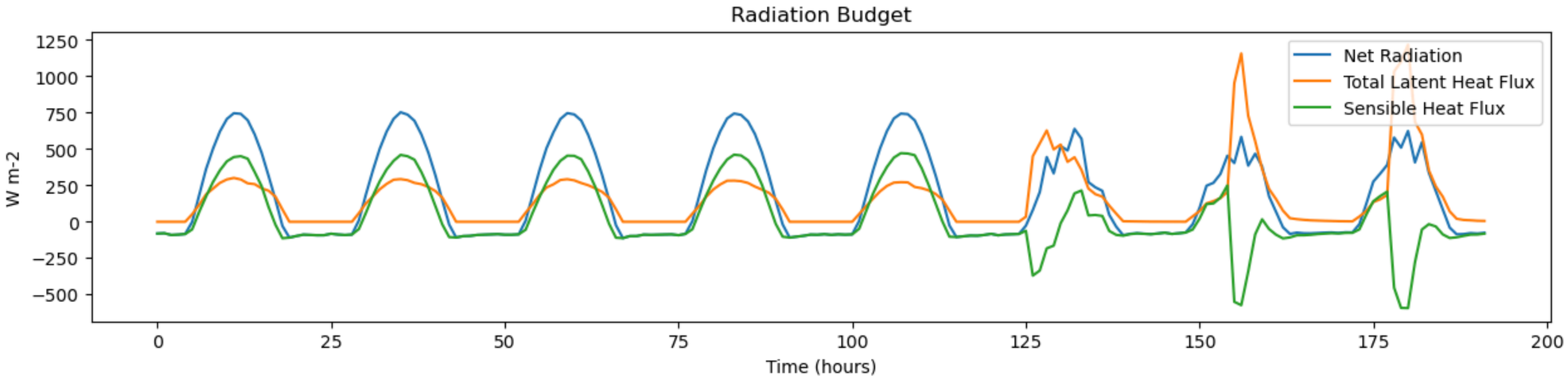
In [ ]:
```
plot_data(forest_data)
```

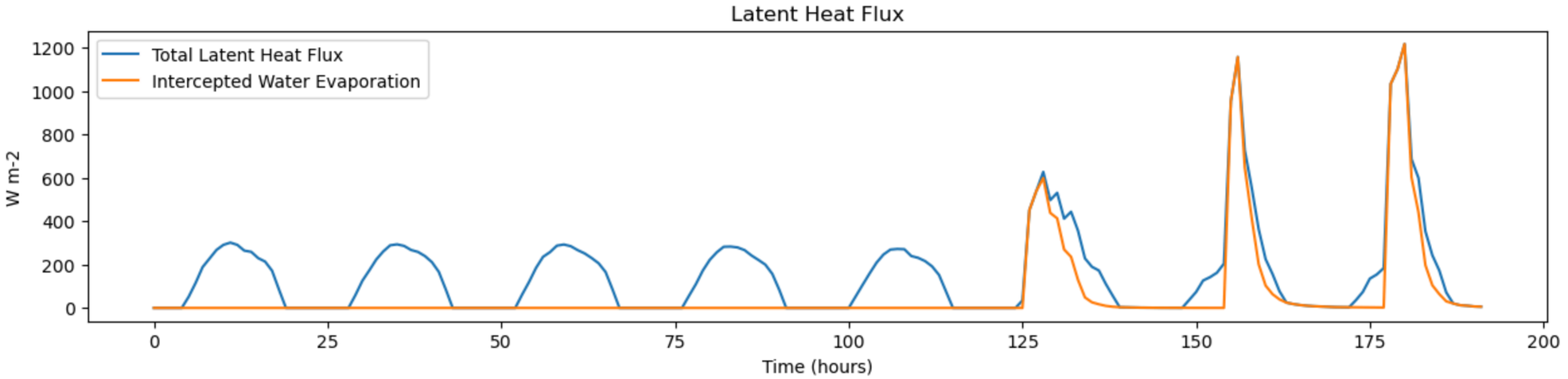
FOREST DATA PLOTS

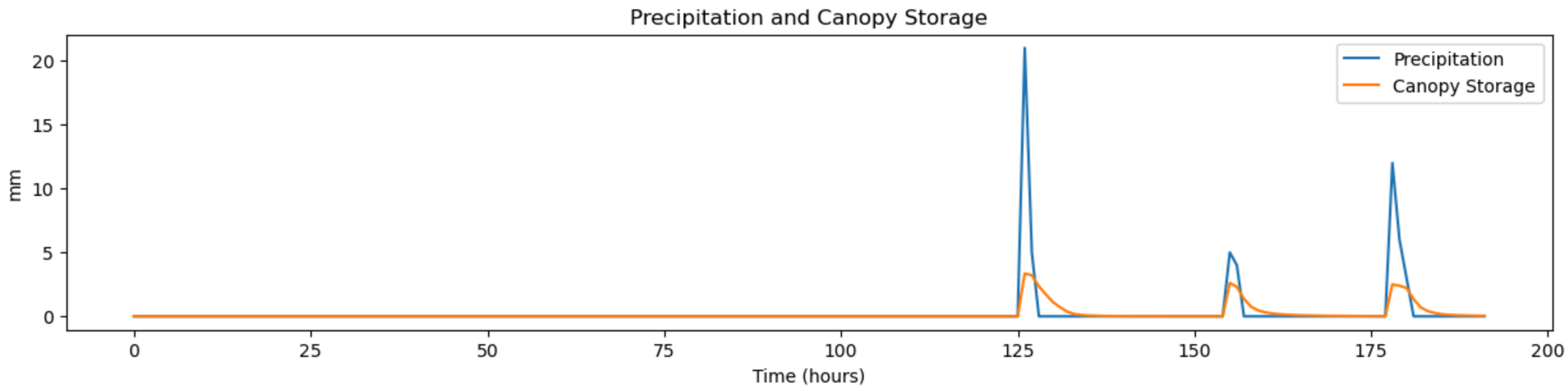


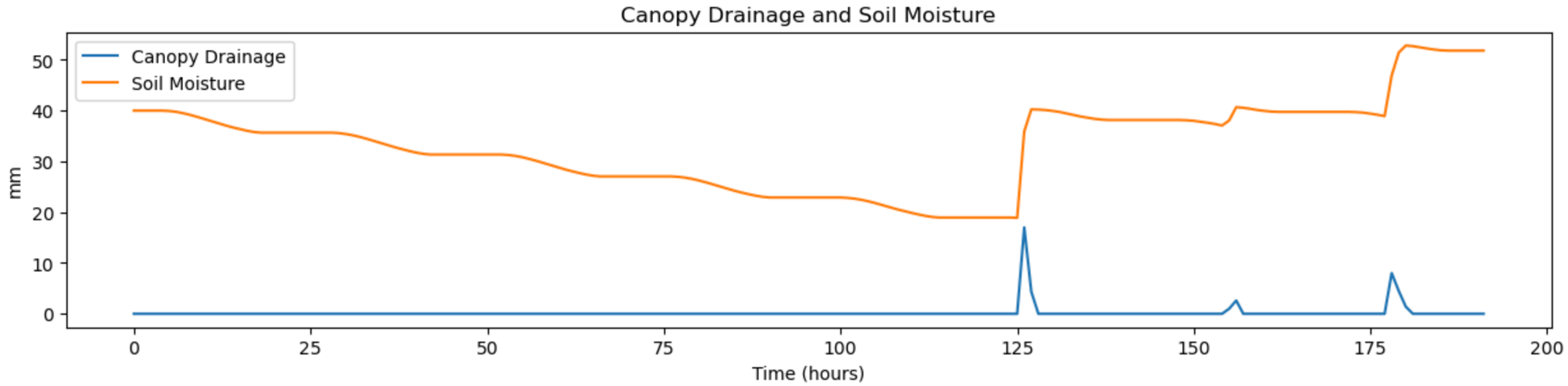


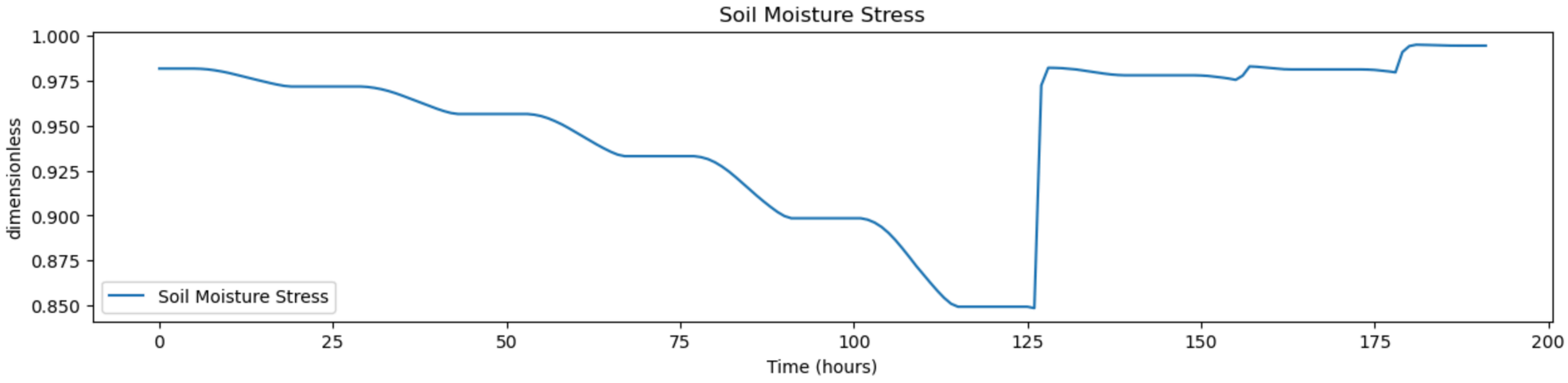



Radiation Budget
Net Radiation
Total Latent Heat Flux
Sensible Heat Flux
W m-2
1250
1000
750
500
250
0
−250
−500
0
25
50
75
100
125
150
175
200
Time (hours)


Latent Heat Flux
Total Latent Heat Flux
Intercepted Water Evaporation
W m-2
1200
1000
800
600
400
200
0
0
25
50
75
100
125
150
175
200
Time (hours)

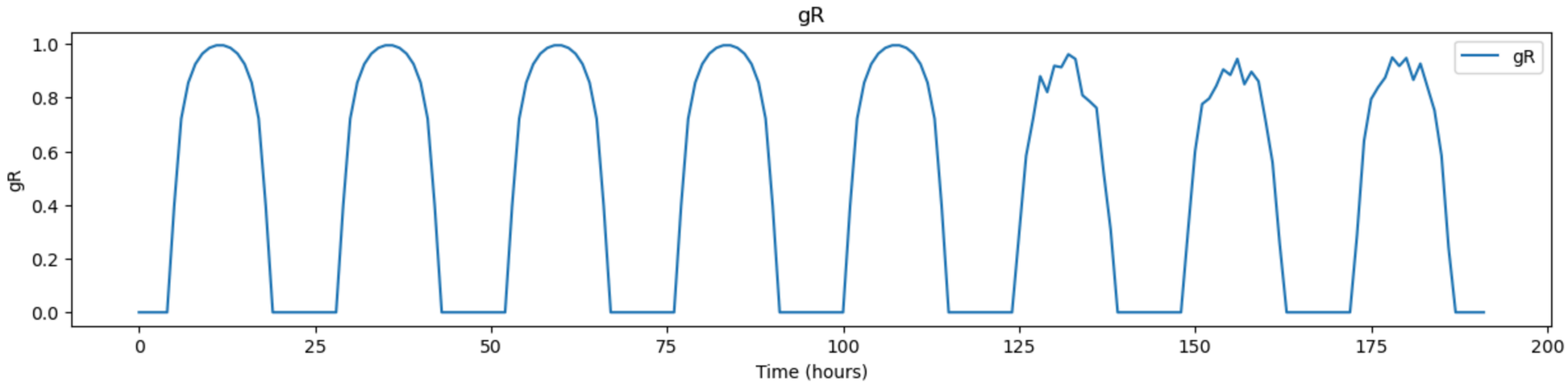

Precipitation and Canopy Storage
Precipitation
Canopy Storage
mm
20
15
10
5
0
0
25
50
75
100
125
150
175
200
Time (hours)

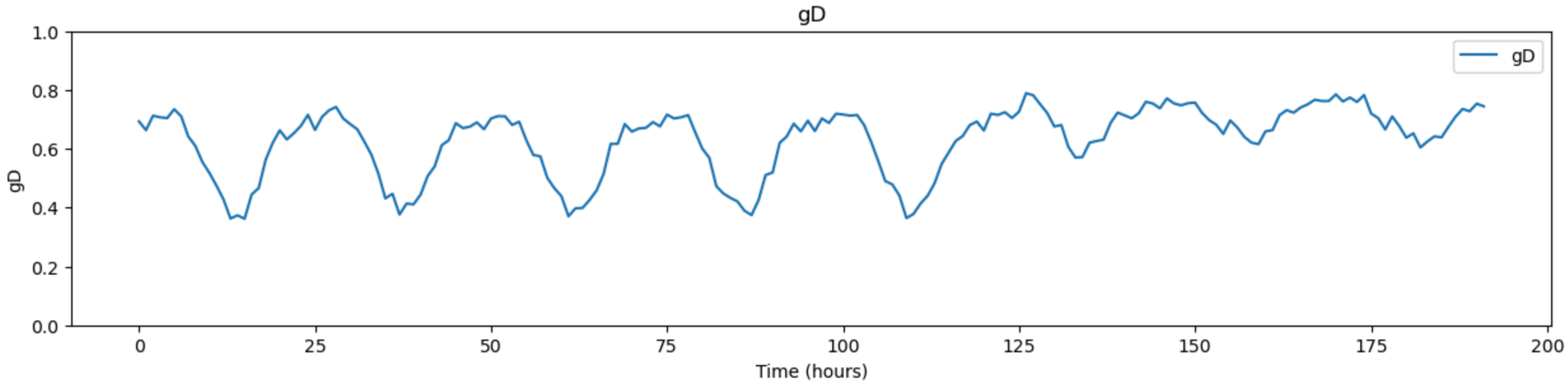


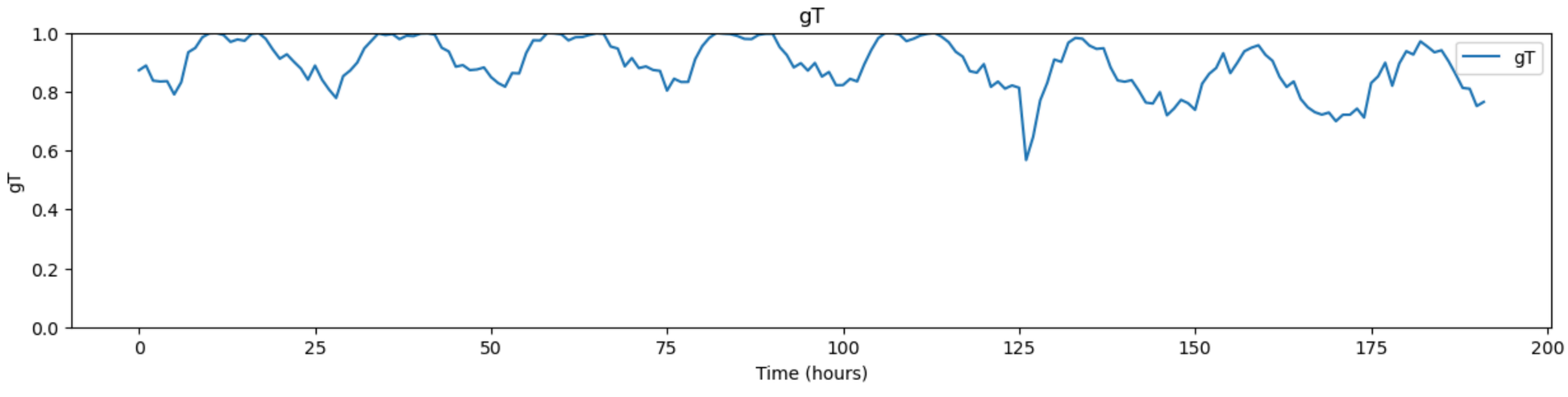


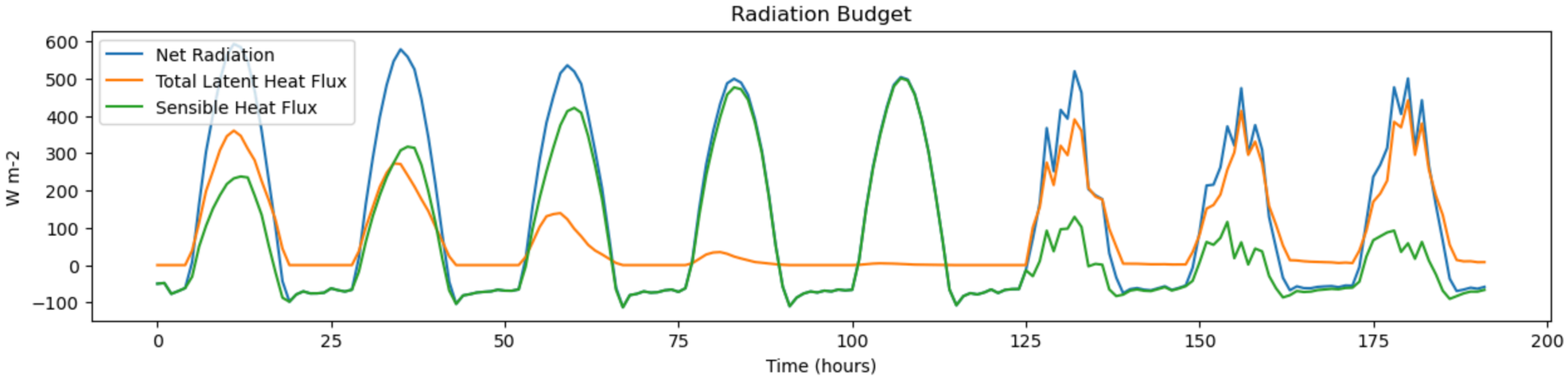
In [ ]:
```
plot_data(grass_data)
```

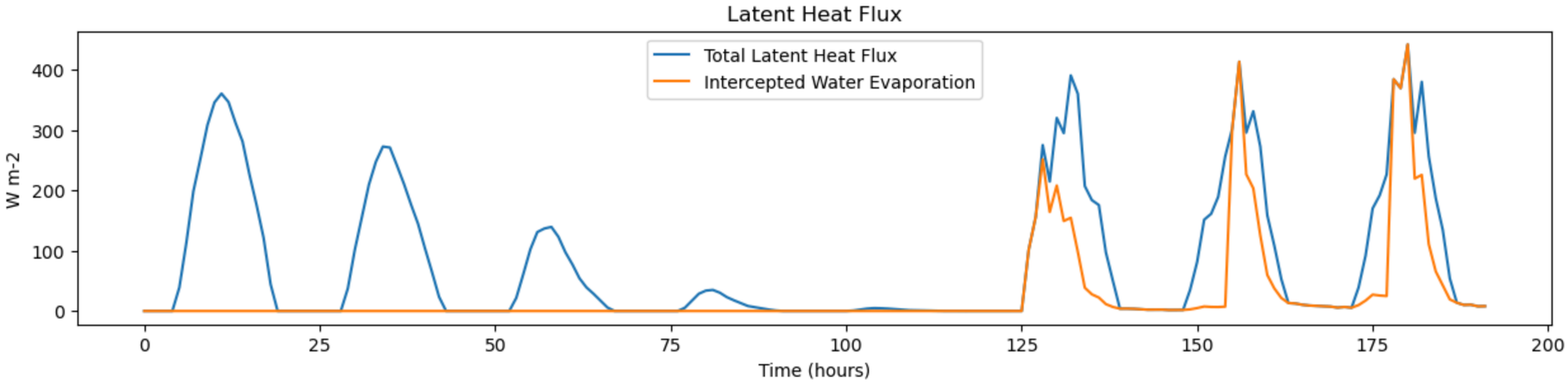
GRASS DATA PLOTS

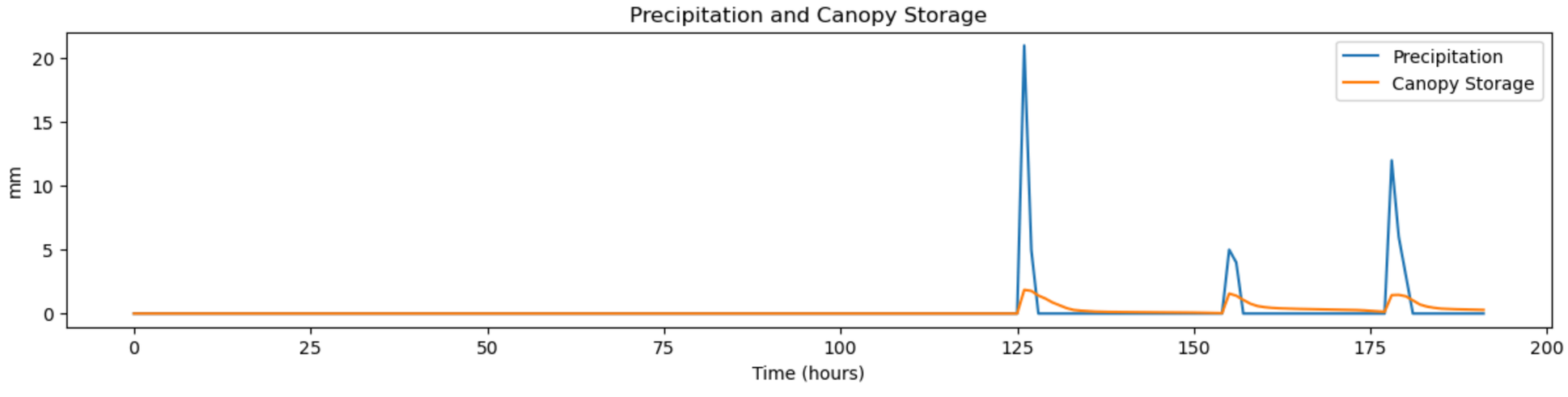


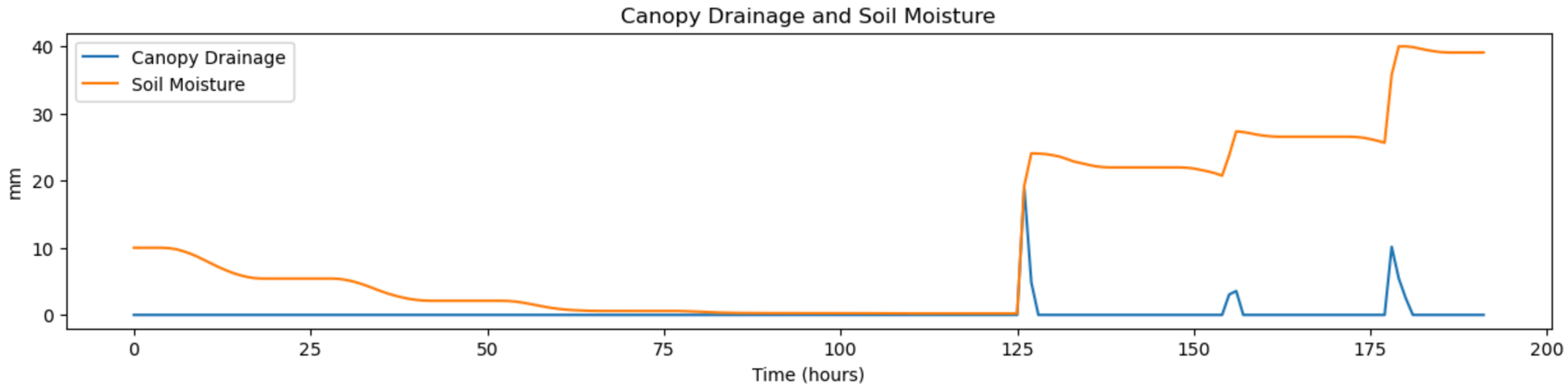


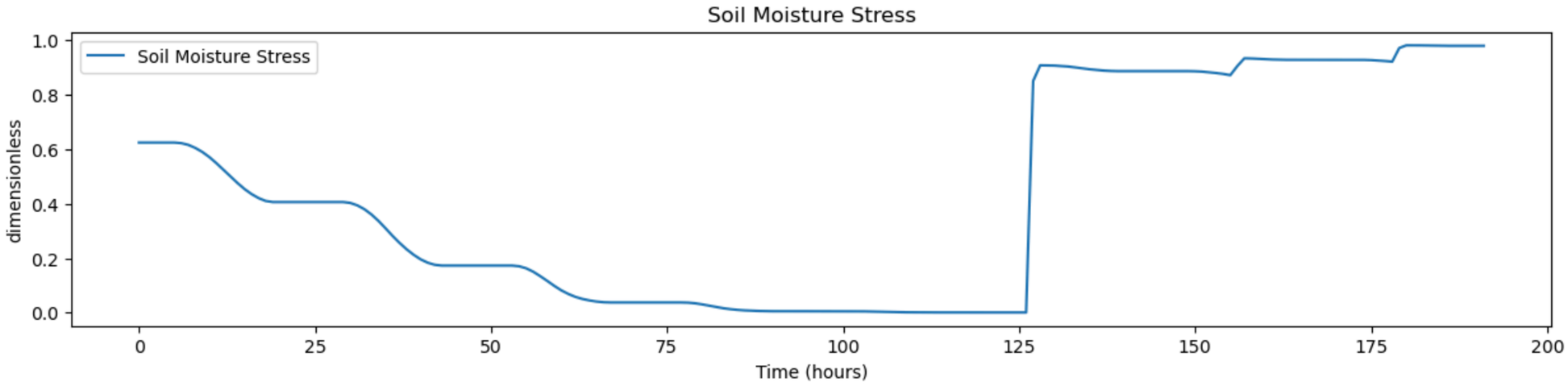



Radiation Budget
Net Radiation
Total Latent Heat Flux
Sensible Heat Flux
W m-2
600
500
400
300
200
100
0
−100
0
25
50
75
100
125
150
175
200
Time (hours)


Latent Heat Flux
Total Latent Heat Flux
Intercepted Water Evaporation
W m-2
400
300
200
100
0
0
25
50
75
100
125
150
175
200
Time (hours)


Precipitation and Canopy Storage
Precipitation
Canopy Storage
mm
20
15
10
5
0
0
25
50
75
100
125
150
175
200
Time (hours)





In [ ]:
```
from IPython.display import display



def display_averages(df, caption):
    # Calculate daily averages, column is in cumulative hours
    df['Day'] = df['Total Hour'] // 24
    # Resetting the index here with as_index=False to keep 'Day' as a column
    daily_averages = df.groupby('Day', as_index=False).mean()

    # Calculate Bowen Ratio and Fractional Contribution
    daily_averages['Bowen Ratio'] = daily_averages['H'] / daily_averages['lambdaE']
    daily_averages['Fractional Contribution of Ei to ET'] = daily_averages['lambdaEi (C=Cactual)'] / daily_averages['lambdaE']

    # Select the columns you want to display
    columns_to_display = ['Day', 'p', 'Rn', 'lambdaE', 'H', 'lambdaEi (C=Cactual)', 'Bowen Ratio', 'Fractional Contribution of Ei to ET', 'SM', 'rs', 'ra', 'Ts']

    # Style the DataFrame for better presentation
```

```
    styled_df = daily_averages[columns_to_display].style.set_table_styles(
        [{
            'selector': 'th',
            'props': [('text-align', 'center')]
        }],
        overwrite=False
    ).set_properties(**{'text-align': 'center'}).format(
        # Specify formatting for each column individually
        {"Day": "{:.0f}", "Rn": "{:.2f}", "lambdaE": "{:.2f}", "H": "{:.2f}",
        "lambdaEi (C=Cactual)": "{:.2f}", "Bowen Ratio": "{:.2f}", "Fractional Contribution of Ei to ET": "{:.3f}"}
    ).set_caption(caption) # Add caption here

    # Display the styled DataFrame
    display(styled_df)

display_averages(forest_data, "Forest")
print(f"Displacement Height: {forest_constants.d:.3f}")
print(f"Aerodynamic Roughness: {forest_constants.z0:.3f}\n")

display_averages(grass_data, "Grass")
print(f"Displacement Height: {grass_constants.d:.3f}")
print(f"Aerodynamic Roughness: {grass_constants.z0:.3f}\n")
```

Forest

	Day	p	Rn	lambdaE	H	lambdaEi (C=Cactual)	Bowen Ratio	Fractional Contribution of Ei to ET	SM	rs	ra	Ts
0	0	0.000000	215.49	123.27	92.22	0.00	0.75	0.000	37.777018	416774.189667	12.013556	290.076573
1	1	0.000000	215.20	122.55	92.66	0.00	0.76	0.000	33.452946	416772.441499	12.087790	289.848145
2	2	0.000000	214.76	122.18	92.58	0.00	0.76	0.000	29.138015	416777.629855	12.055120	290.293996
3	3	0.000000	214.47	118.01	96.46	0.00	0.82	0.000	24.919212	416780.248567	12.093480	290.126120
4	4	0.000000	214.59	112.24	102.35	0.00	0.91	0.000	20.871574	416784.517290	11.916224	290.062097
5	5	1.083333	130.91	194.29	-63.38	133.08	-0.33	0.685	33.801783	416780.508745	13.247866	284.826755
6	6	0.375000	125.91	209.98	-84.07	152.58	-0.40	0.727	38.945591	416773.887996	13.020476	284.309120
7	7	0.875000	149.68	255.95	-106.27	203.50	-0.42	0.795	46.620085	416773.317612	12.607952	284.137553

```
Displacement Height: 14.644
Aerodynamic Roughness: 1.607
```

Grass

	Day	p	Rn	lambdaE	H	lambdaEi (C=Cactual)	Bowen Ratio	Fractional Contribution of Ei to ET	SM	rs	ra	Ts
0	0	0.000000	169.81	130.20	39.61	0.00	0.30	0.000	7.599797	416770.553576	75.657873	291.158447
1	1	0.000000	160.70	94.17	66.53	0.00	0.71	0.000	3.629719	416863.377612	76.125380	292.212959
2	2	0.000000	146.75	43.12	103.63	0.00	2.40	0.000	1.235872	417432.562579	75.919630	294.805281
3	3	0.000000	136.96	9.39	127.57	0.00	13.58	0.000	0.392093	421726.825263	76.161215	296.117699
4	4	0.000000	136.81	1.27	135.53	0.00	106.68	0.000	0.231942	455523.729127	75.044908	296.131188
5	5	1.083333	105.51	118.46	-12.95	58.59	-0.11	0.495	16.959334	437809.463349	83.431197	284.544278
6	6	0.375000	102.37	119.68	-17.31	62.04	-0.14	0.518	24.300060	416724.683143	81.999163	283.841820
7	7	0.875000	121.94	137.71	-15.77	86.13	-0.11	0.625	33.797140	416721.828397	79.401210	283.992915

```
Displacement Height: 0.077
Aerodynamic Roughness: 0.013
```

In [ ]:

```python
# D.2 Canopy Transpiration.
#This section currently unused. Section D.1 used instead.

def calc_lambdaET_dry_D2(delta, Rn, ra, rs, psy_const, rho_a, Dref):
    """
    Calculate transpiration rate
    delta: gradient of the saturation vapor pressure curve in kPa/°C
    Rn: net radiation in W/m^2
    ra: aerodynamic resistance in s/m
    psy_const: psychrometric constant in kPa/°C
    rho_a: air density in kg/m^3
    Dref: observed VPD at reference level above canopy in kPa
    return: transpiration rate in W/m^2
    """
    A = Rn # Available energy = net radiation problem statement D.2.1
    lambdaET_dry = delta * A + (rho_a * cp * Dref / ra) / (delta + psy_const * (1 + ra / rs)) # Eq 22.18 TH, problem statement D.2.1
    return lambdaET_dry

def calc_H_D2(Rn, lambdaET_dry):
    """
    Calculate turbulent sensible heat flux from net radiation, evaporation rate, and heat flux into the ground.
    R_n: Net radiation in W/m^2.
    E: Evaporation rate in m/s.
    G: Heat flux into the ground in W/m^2.
    return: Turbulent sensible heat flux in W/m^2.
    """
    H = Rn - lambdaET_dry # Given in problem statement D.2.2
    return H
```

ANSWERS TO QUESTIONS

i. the relative differences in values of daily average Bowen Ratio,

[Hints. What are the differences in the Bowen ratio for forest and grass? How does this value changes as the soil dries?]

	Day	Rn	lambdaE	H	lambdaEi (C=Cactual)	Bowen Ratio	Fractional Contribution of Ei to ET	SM	rs	ra
0	0	215.49	123.27	92.22	0.00	0.75	0.000	37.777018	416774.189667	12.013556
1	1	215.20	122.55	92.66	0.00	0.76	0.000	33.452946	416772.441499	12.087790
2	2	214.76	122.18	92.58	0.00	0.76	0.000	29.138015	416777.629855	12.055120
3	3	214.47	118.01	96.46	0.00	0.82	0.000	24.919212	416780.248567	12.093480
4	4	214.59	112.24	102.35	0.00	0.91	0.000	20.871574	416784.517290	11.916224
5	5	130.91	194.29	-63.38	133.08	-0.33	0.685	33.801783	416780.508745	13.247866
6	6	125.91	209.98	-84.07	152.58	-0.40	0.727	38.945591	416773.887996	13.020476
7	7	149.68	255.95	-106.27	203.50	-0.42	0.795	46.620085	416773.317612	12.607952

```
Displacement Height: 14.644
Aerodynamic Roughness: 1.607
```

Grass

	Day	Rn	lambdaE	H	lambdaEi (C=Cactual)	Bowen Ratio	Fractional Contribution of Ei to ET	SM	rs	ra
0	0	169.81	130.20	39.61	0.00	0.30	0.000	7.599797	416770.553576	75.657873
1	1	160.70	94.17	66.53	0.00	0.71	0.000	3.629719	416863.377612	76.125380
2	2	146.75	43.12	103.63	0.00	2.40	0.000	1.235872	417432.562579	75.919630
3	3	136.96	9.39	127.57	0.00	13.58	0.000	0.392093	421726.825263	76.161215
4	4	136.81	1.27	135.53	0.00	106.68	0.000	0.231942	455523.729127	75.044908
5	5	105.51	118.46	-12.95	58.59	-0.11	0.495	16.959334	437809.463349	83.431197
6	6	102.37	119.68	-17.31	62.04	-0.14	0.518	24.300060	416724.683143	81.999163
7	7	121.94	137.71	-15.77	86.13	-0.11	0.625	33.797140	416721.828397	79.401210

```
Displacement Height: 0.077
Aerodynamic Roughness: 0.013
```

The Bowen Ratio is initially higher in forest than in grass (the soil moisture for forest is also initially higher). As the soil dries, the Bowen Ratio goes up in forest but goes up significantly more in grass. Once the soil moisture starts increasing, the bowen ratio decreases in grass, but decreases at a greater magnitude in forest. Overall, the bowen ratio of grass has a much wider range and is much more volatile than the bowen ratio of forest.

The Bowen Ratio is a ratio of sensible heat flux to latent heat flux. Forest and grass crops have a significantly different LAI and canopy height, which lead to differing displacement heights and aerodynamic roughness between the two surfaces. These determine the aerodynamic resistance (r_a) of the surface, which is one of the parameters driving the calculation of latent heat flux. Forest has a high LAI and aerodynamic roughness, which leads to a relatively lower aerodynamic resistance. A larger aerodynamic resistance drives more ET, which lowers soil moisture.

As seen in the table above, the bowen ratio for grass starts out lower than forest because more evapotranspiration (ET, represented as latent heat flux) is occurring. But as more evapotranspiration occurs, the soil moisture nears 0 and the stomatal resistance increases. This causes ET to lower, so the energy from the incoming solar radation is converted to sensible heat flux, and the bowen ratio rises.

On days 5-7 the incoming solar radation, and thus net radiation, decrease, which causes a loss of sensible heat over these days. Precipitation occurs over these days, so the soil moisture is replenished, lowering the stomatal resistance of both forest and grass, and raising total ET and lowering the Bowen ratio. The Forest has a higher net radiation during this time which is driven by lower longwave radation emitted from the surface due to lower surface temperatures. This leads to a relatively higher sensible heat than grass and a higher bowen ratio.

The ratio of the sensible heat flux to the latent heat flux is called the *Bowen Ratio*, β. If the instantaneous values of sensible and latent heat are H and λE, respectively, the instantaneous value of the Bowen Ratio is therefore given by:

$$\beta = \frac{H}{\lambda E} \qquad (4.6)$$

$$r_a = r_a^V = r_a^H = \frac{1}{k^2 u_m} \ln\left(\frac{z_m - d}{z_0}\right) \ln\left(\frac{z_m' - d}{z_0/10}\right)$$

$$\lambda E_T = \frac{\Delta A + \left(\rho_a c_p D_{ref}\right)/r_a}{\Delta + \gamma\left(1 + r_s/r_a\right)}$$

ii. the ratio of daily average fractional contribution to daily average total evaporation that arises from the evaporation of intercepted water you find for forest and grass [if interception was considered].

Forest has a higher LAI that leads to a lower aerodynamic resistance, which contributes to a higher proportion of evaporation of intercepted water (Ei). Intuitively this makes sense - forested areas will catch more precipitation in their canopy than grassy areas because there is more area for the water to be caught.